

Introduzione all'ingegneria del software

Ingegneria del Software Avanzata

CdLM in Ingegneria Informatica e dell'Automazione

A.A. 2025/2026

Marco Alberti (lbmrnc@unife.it)

Sommario

- Che cos'è l'ingegneria del software
- Perché esiste l'ingegneria del software
- Che cosa può e non può fare l'ingegneria del software
- Che cosa fa l'ingegnere del software
- Relazione fra ingegneria del software e altri campi dell'informatica

Che cos'è l'ingegneria del software?

- Il software è l'insieme dei programmi e della documentazione necessari per il funzionamento di un sistema di elaborazione delle informazioni.
- Una fra le varie definizioni: l'ingegneria del software è l'«applicazione di un approccio sistematico, disciplinato e quantificabile allo sviluppo, all'operatività e alla manutenzione del software» (“IEEE Standard Glossary of Software Engineering”, 1990)
- L'approccio ingegneristico ha un costo aggiuntivo rispetto alla mera programmazione; risulta vantaggioso nella costruzione di sistemi software:
 - grandi e complessi (troppo per essere compresi da una sola persona)
 - prodotti da squadre
 - co-esistenti in diverse versioni
 - in funzione per anni o decenni
 - che subiscono evoluzioni

Un po' di storia

- **Anni '50:**
 - Il software implementava soluzioni a problemi ben compresi (es. calcolo numerico)
 - L'utente e il programmatore erano spesso la stessa persona
 - Ingegneria del software = programmazione
- **Anni '60:**
 - Calo dei costi dell'hardware, maggiore diffusione dei computer
 - Software applicato a problemi vari, più complessi, non sempre ben compresi
 - Utente != programmatore

Crisi del software

- difficoltà di comunicazione fra utenti e programmatori, requisiti del software non chiari
- linguaggi e tecniche di programmazione non adatti a sistemi complessi
- parti dei sistemi software fortemente accoppiate:
 - necessità di coordinazione continua fra i programmatori
 - difficoltà nella sostituzione di chi abbandonava i progetti
 - la modifica di una parte influenzava l'intero sistema
- Risultato: sistemi poco soddisfacenti, poco affidabili, che non rispettavano i tempi e i costi previsti
- Coniata l'espressione «crisi del software» (NATO Software Engineering Conference, 1968)
- Nascita dell'ingegneria del software come disciplina a sé stante

Contributi dell'ingegneria del software

A partire dagli anni 60, la ricerca sull'ingegneria del software ha prodotto:

- Linguaggi e strumenti
- Approcci rigorosi (formali) per
 - specifica
 - convalida
dei sistemi software
- Standard
- Metodologie pratiche, organizzative e gestionali

L'approccio ingegneristico non risolve tutti i problemi

- Le difficoltà:
 - non esiste un linguaggio universalmente accettato per definire i requisiti
 - le tecniche formali sono poco sviluppate e di applicabilità non banale a sistemi complessi
 - (quasi) ogni nuovo prodotto pone problemi nuovi (se così non fosse, si userebbe una libreria anziché produrre un nuovo software)
- Intuito ed esperienza giocano ancora un ruolo fondamentale
- La progettazione e lo sviluppo del software richiedono sforzo intellettuale, creatività e tempo

Ruolo dell'ingegnere del software

Ideare e scrivere programmi, ma anche

- comprendere i requisiti e tradurli in specifiche precise
- progettare (architettura dei componenti)
- operare a diversi livelli di astrazione
- comunicare con gli (altri) sviluppatori, gli utenti, i clienti
- stimare e rispettare tempi e costi

Fasi della produzione di software

- preliminare: studio di fattibilità (stima di benefici e costi di diversi scenari e soluzioni)
- analisi e specifica dei requisiti
- progettazione
- sviluppo e verifica dei componenti
- integrazione e test di sistema
- messa in produzione e manutenzione

Analisi e specifica dei requisiti

- Obiettivo: identificare e documentare i requisiti che il sistema software dovrà rispettare (che cosa)
- In caso di sistemi innovativi, richiede ampia interazione fra cliente e ingegnere
- Fase critica e complessa
- Il documento dei requisiti dovrebbe
 - essere comprensibile per l'utente finale, in modo che possa convalidarlo
 - poter portare alla creazione del manuale utente e di casi di test

Progettazione

- Obiettivo: progetto di un particolare sistema che soddisfi i requisiti (come)
- Divisa in due fasi:
 - Progetto architettuale: organizzazione globale del sistema in componenti di alto livello e loro interazioni
 - Progetto dettagliato: scomposizione a livelli di dettaglio sempre maggiori, fino a una specifica diretta dei requisiti dei singoli componenti (a livello, ad esempio, di funzione o di metodi di una classe)

Sviluppo e verifica dei componenti

- Produzione del codice sorgente secondo il progetto dettagliato del sistema
- Verifica che i singoli moduli prodotti rispettino la loro specifica
 - Analisi statica, code review
 - Unit testing

Integrazione e verifica di sistema

I singoli componenti prodotti vengono

- assemblati
- integrati
 - per esempio, inizializzati in modo da interagire con gli altri componenti del sistema
- testati come sistema unico
 - casi di test «end-to-end» derivati, esplicitamente o meno, dalla specifica dei requisiti

Consegna e manutenzione

Quando il sistema ha superato tutti i test viene consegnato all'utente, ad esempio:

- installato nel sistema dell'utente
- reso disponibile per lo scaricamento e l'installazione
- installato e messo in funzione su un server web

Successivamente, si rende necessaria la manutenzione (modifica) del sistema:

- per correggere i difetti che si manifestano
- per adattare il sistema a cambiamenti dei requisiti (evoluzione)

Interazione dell'ingegneria del software con altre discipline

Fin dalla nascita, l'ingegneria del software ha interagito bilateralmente con altre discipline informatiche:

- Linguaggi di programmazione
- Sistemi operativi
- Basi di dati
- Metodi formali
- Intelligenza artificiale

Ingegneria del software e linguaggi di programmazione

- Forte influenza dell'ingegneria del software sui linguaggi di programmazione: ad esempio,
 - Costrutti per programmazione modulare
 - Costrutti per la gestione delle eccezioni
 - Separazione tra interfaccia e implementazione
- Influenza inversa:
 - Linguaggi per specifiche di progetto, adatti all'implementazione in un linguaggio di programmazione
 - (ad esempio specifiche UML facilmente implementabili come gerarchie di classi in vari linguaggi orientati agli oggetti)

Ingegneria del software e sistemi formali (modelli matematici)

- Modelli formali utilizzati in ingegneria del software (specifiche e modelli)
 - automi a stati finiti
 - automi a pila
 - reti di Petri
 - logica matematica
- Inversamente: sviluppo di nuovi metodi formali
 - specifiche algebriche
 - logica temporale

Modello a strati del software

1. **Requisiti di business** – obiettivi di valore per l'organizzazione.
2. **Requisiti utente** – funzionalità percepite dal punto di vista dell'utente finale.
3. **Requisiti di sistema** – vincoli tecnici, di performance, di sicurezza, ecc.
4. **Progetto** – modello strutturale del sistema (componenti, interfacce, pattern architetturali)
5. **Codice** – sorgente scritto nei linguaggi di programmazione scelti
6. **Installazione** - artefatti di installazione nel loro ambiente operativo
7. **Processi** - insieme delle esecuzioni del software (funzione input → output)

Il valore è prodotto dai processi.

Ogni strato è una descrizione dell'insieme dei processi

Sviluppo e convalida

- Finestra mobile su tre strati contigui
 - perché (es. requisiti di sistema)
 - cosa (es. progetto)
 - come (es. codice)
- Ogni strato s specifica (motiva) quello direttamente inferiore, che è *sviluppato* e deve essere *convalidato* rispetto ad s .
- Le fasi dell'ingegneria del software si inquadrano in questo schema: sviluppo e convalida.

Agenti nel processo software

- Certe attività di sviluppo possono essere eseguite tramite computazione (es. compilazione da sorgente a eseguibile).
- Altre richiedono *sintesi linguistica* da parte di un *agente*.
- Fino a pochi anni fa, solo gli umani erano capaci di sintesi linguistica efficace.
- Dal 2022 in poi, esistono agenti artificiali in grado di sostituire gli umani in alcune attività di sintesi linguistica; in particolare, nella generazione di codice.

Ingegneria del software nell'era dell'AI

- Il modello a strati rimane (per ora) invariato
- AI può ridurre il costo di alcune attività (es. codifica, revisione)
- Gli LLM attuali necessitano di contesto *esplicito*, il che rende ancora più importanti gli strati al di sopra del codice, e altri vincoli tradizionalmente impliciti
- LLM come "compilatore" da requisiti a codice?
 - LLM non deterministici (a differenza dei veri compilatori o interpreti)
 - requisiti ambigui (a differenza del codice nei linguaggi di programmazione)
- Chi produce software è *responsabile* dei processi (coppie input/output)